Week 14 (Days 92-98): The transformer from scratch

Status: written in full (Phase 4). Titles match `SYLLABUS.md`. Every number below was printed by a script in `code/week-14/` on this box (Python 3.13.5, torch 2.15.0+cpu, 8 CPU threads, no GPU). Wall-clock times were measured while other jobs shared the CPU. Treat them as rough and re-time on your own machine.

Week goals

- Implement multi-head self-attention with causal and padding masks and match PyTorch to float precision.
- Understand why attention needs positional information; implement sinusoidal encodings and RoPE and verify RoPE's relative-position property.
- Assemble a pre-LN transformer block, count its parameters exactly, and compare pre-LN with post-LN.
- Write and train a tiny GPT on CPU and beat a bigram baseline.
- Write byte-level BPE from scratch, compare it with a pretrained tokenizer, and speed up the encoder in Rust.
- Run a miniature scaling experiment and read the published scaling-law papers critically.
- Train the GPT on BPE tokens (weekly project).

```
flowchart LR
  A[Day 92 MHA + masks] --> B[Day 93 positions]
  B --> C[Day 94 block]
  C --> D[Day 95 tiny GPT]
  D --> E[Day 96 BPE + Rust]
  E --> F[Day 97 scaling]
  F --> G[Day 98 project + quiz]
```

Compute note (CPU vs GPU). Everything here trains on a CPU in minutes, with models under a million parameters. On a single modern GPU the same code (with `.cuda()`) would run roughly one to two orders of magnitude faster per step, which would let you train ~10-100x larger models in the same time. The *ideas* do not change with scale; the sample quality does.

Datasets: Tiny Shakespeare (1,115,394 characters, cached by `code/common.py`; 90/10 train/val split by position). Day 96 also downloads the SmolLM2-135M tokenizer from the Hugging Face Hub (`HuggingFaceTB/SmolLM2-135M`, Apache-2.0, download verified on this box) into `data/cache/hf`.

Shared code: `code/week-14/_gpt.py` (the GPT used through Day 119) and `code/week-14/_bpe.py` (the tokenizer). Rust: `code/week-14/rust_bpe` (`maturin develop --release`).

Day 92: Self-attention, multi-head attention, masking

Review today: Day 91 (+1), Day 89 (+3), Day 85 (+7), Day 71 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

From NumPy to PyTorch. Day 91 wrote attention in NumPy. Today we write the batched PyTorch version used in real models. Its shapes are worth memorising: the input x is (B, T, D); one linear layer produces Q, K and V together, (B, T, 3D); we reshape to (B, T, 3, h, D/h) and permute to three tensors of shape (B, h, T, D/h). Scores are (B, h, T, T). The output is transposed back to (B, T, h, D/h), reshaped to (B, T, D) and projected by W_O.

Self-attention vs cross-attention. In self-attention Q, K and V all come from the same sequence. In cross-attention (Day 81's decoder looking at the encoder) Q comes from one sequence and K, V from another. A decoder-only GPT uses only causal self-attention.

Masks. A mask says which (query, key) pairs are allowed. Two kinds: - **Causal mask** (lower-triangular, T x T): a token may only attend to itself and earlier tokens. This is what makes next-token training possible on a whole sequence in parallel: position t predicts token t+1 without seeing it. - **Padding mask** (B x T): in a batch of sequences of different lengths, shorter ones are padded. Keys at padding positions must get zero weight, or the padding changes the real tokens' outputs. Combine them with a logical AND and broadcast to (B, 1, T, T) so the same mask applies to every head. Masked scores are set to -inf before the softmax.

The NaN trap. If a query row is masked entirely (a padding query with no allowed keys), its softmax is 0/0 = NaN, which then poisons the loss. Real code either never lets a row be fully masked (the causal diagonal is always allowed) or replaces those rows with zeros and excludes padding positions from the loss.

Cost. The score matrix is T x T per head. Time and memory grow quadratically with sequence length. That single fact drives Day 111 (KV cache) and Day 115 (efficient attention).

Block 2: Worked example (45 min)

Run `code/week-14/day92_mha_masking.py` (a few seconds). **Verified:** - Causal multi-head attention (B 2, T 6, D 16, 4 heads) vs `nn.MultiheadAttention` with the same weights: max diff **6.0e-08**. Weights above the diagonal: all zero (**True**). - Padding: the second sequence has length 4. The maximum weight on its padded keys is **0.0**, and its real positions match running the length-4 sequence alone with max diff **0.0e+00**. That is the definition of a correct padding mask. - A fully masked row gives NaN: **True**. - `F.scaled_dot_product_attention(..., is_causal=True)` vs mine: max diff **6.0e-08**. - Parameters: 4.D^2 = **1,024** = `nn.MultiheadAttention(bias=False)`. - Naive attention time (4 heads, d_head 64), with the size of the fp32 score matrix:

T	time (ms)	score matrix (MiB)
256	0.7	1.0
512	3.0	4.0
1024	9.6	16.0
2048	40.4	64.0

Doubling T quadruples the memory exactly. Time grows by roughly 3-4x per doubling at the larger sizes; at small T, fixed overheads dominate. An earlier run of the same script while the CPU was shared gave different times (13.5 ms and 71.0 ms for T = 1024 and 2048), so trust the ratios more than the absolute values.

Block 3: Coding lab (2h)

1. (45 min) Write `mha(x, Wqkv, Wo, h, mask)` without looking at the script. Match `nn.MultiheadAttention` to 1e-6. Note that PyTorch's `attn_mask` uses `True` = *blocked*, the opposite of our convention.
2. (45 min) Build a padded batch of three sequences (lengths 6, 3, 5) and prove each real output equals the unpadded computation.
3. (30 min) Write a `safe_softmax` that returns zeros for fully masked rows, and test it on a batch where one sequence is all padding.

Block 4: Practice problems (1h)

1. x is (8, 100, 512), 8 heads. Shape of the scores? *Answer: (8, 8, 100, 100).*
2. Memory of those scores in fp32? *Answer: 8.8.100.100.4 bytes = 2.56 MB.*
3. Why does the causal mask allow training on all positions at once? *Answer: each position's output depends only on earlier tokens, so all T next-token losses are valid simultaneously.*
4. Why must padding keys be masked but padding queries need not be (if excluded from the loss)? *Answer: keys affect other tokens' outputs; a padding query only affects its own output, which is ignored.*
5. Parameters of multi-head attention with D = 768, with biases on all four projections? *Answer: 4.768^2 + 4.768 = 2,362,368.*

Block 5: Review (1h)

Self-check: - [ ] I can write every shape in a batched MHA forward pass. - [ ] I can combine causal and padding masks and know which way round PyTorch's mask convention goes. - [ ] I can explain the NaN trap and two fixes.

Extra notes: what different heads do

Splitting D into h heads does not add parameters: 4D^2 either way. What changes is that each head computes its own softmax, so each can produce a different weighting of the past. With one head of size D, a token must blend everything it wants into one distribution. With 4 heads of D/4, one head can attend to the previous token while another attends to the most recent occurrence of the same token, which is the "induction head" pattern found in trained language models. The cost is that each head's dot products live in only D/h dimensions, so each head is less expressive on its own. In practice h is chosen so that D/h is 64 or 128. Multi-query and grouped-query attention (Day 111) go one step further and share K and V across heads, which saves cache

memory at inference time with little quality loss.

Going deeper (optional)

Read the PyTorch documentation for `torch.nn.functional.scaled_dot_product_attention` and note which backends it can dispatch to.

Interview-style question (15 min, written)

"How does a padding mask differ from a causal mask, and what goes wrong if you forget the padding mask?" Causal blocks future positions; padding blocks fake positions. Without the padding mask, a sequence's outputs change depending on how much padding it was batched with, so results depend on batch composition.

Common mistakes / how to debug

- Reshaping (B, T, 3D) straight to (B, h, T, ...) without the permute: the heads silently mix positions.
- Using PyTorch's boolean mask with the wrong polarity.
- Masking with a finite -1e4 in fp16, which may not be "minus infinity" enough after the scale.
- Forgetting the 1/sqrtd_head scale (Day 91).

What to do next

Day 93 gives attention a sense of order.

Day 93: Positional encodings (sinusoidal, RoPE)

Review today: Day 92 (+1), Day 90 (+3), Day 86 (+7), Day 72 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The problem. Attention is a weighted average whose weights depend on content only. Permute the input tokens and the outputs are permuted in the same way: "dog bites man" and "man bites dog" contain the same information as far as attention can tell. A language model needs order.

Learned absolute positions. Add a trainable vector p_t to the token embedding at position t. This is what GPT-2 does and what our `_gpt.py` uses. It is simple and works well within the trained context length, but it cannot extrapolate beyond it: position 129 never had a vector.

Sinusoidal encodings (Vaswani et al., 2017). A fixed vector with pe[t, 2i] = sin(t / 10000^{2i/d}) and pe[t, 2i+1] = cos(t / 10000^{2i/d}). Each pair of dimensions is a clock running at a different frequency, from fast (period 2pi) to very slow. Because of the angle-addition identities, the encoding of t + k is a fixed linear function (a rotation) of the encoding of t, so offsets are easy to represent. A useful consequence: the dot product pe[t].pe[t+k] depends only on k, not on t.

RoPE (rotary position embedding) (Su et al., 2021, "RoFormer"). Instead of adding positions to the input, rotate each pair of dimensions of q and k by an angle proportional to the position: for position m and pair i, rotate by m.?_i with ?_i = 10000^{-2i/d}. The dot product of a rotated query at m and a rotated key at n equals the dot product of the originals with a rotation by (m - n). So attention scores depend on *relative* position only, and values are untouched. RoPE is used by most current open LLMs, including the SmolLM2 models used from Day 99 (their config files list rope parameters).

Other options, briefly. Relative position biases (T5) add a learned bias per distance bucket to the scores. ALiBi adds a fixed linear penalty proportional to distance. Both aim at better length extrapolation.

Block 2: Worked example (45 min)

Run `code/week-14/day93_positional.py` (under a second). **Verified:** - Without positions, permuting the inputs just permutes the outputs: **True**. - Sinusoidal encoding for 64 positions x 16 dims: values within [-1, 1]: **True**. After adding it, attention is no longer permutation-equivariant: **False**, as intended. - Dot products pe[10].pe[10+k] for k = 0, 1, 2, 4, 8, 16: **8.0, 7.49, 6.37, 5.56, 4.7, 4.21**. They fall with distance (not monotonically in general, but here they do over this range). Same offset, different start: pe[10].pe[14] = **5.5597** and pe[40].pe[44] = **5.5597**: identical, as the identity predicts. - RoPE: q.k with offset 3 at positions (3, 0),

(10, 7), (100, 97), (1000, 997): **-0.36481, -0.36481, -0.36481, -0.36483**. Equal up to float32 rounding, which grows slightly with large angles. - RoPE preserves norms (it is a rotation): **True**. - For identical all-ones q and k, the RoPE dot product by distance 0, 1, 4, 16, 64, 256: **16.0, 14.97, 11.12, 8.43, 9.82, 6.36**. It tends to decay with distance but not monotonically (64 is higher than 16), so "RoPE decays with distance" is a tendency, not a guarantee.

Block 3: Coding lab (2h)

1. (45 min) Implement `rope(x, pos)` for a tensor of shape (B, h, T, d) and add it to the attention in `_gpt.py` as an option (`pos="rope"`), removing the learned position table.
2. (45 min) Train the Day-95 GPT (next day's script) for 500 steps with learned positions and with RoPE. Compare val loss. Then evaluate both on contexts of 256 tokens (twice the training length): which degrades less?
3. (30 min) Plot pe for 64 positions as an image (positions x dimensions) and explain the stripes.

Block 4: Practice problems (1h)

1. Why can't learned absolute positions handle position 200 if ctx = 128? *Answer: there is no trained vector for it.*
2. What is the period of the first sinusoidal pair (i = 0)? *Answer: 2pi ~= 6.28 positions.*
3. RoPE rotates q and k. Why not v? *Answer: position should influence which tokens attend to which, not the content that is passed on.*
4. Show that rotation preserves dot products of vectors rotated by the same angle. *Answer: R is orthogonal, (Ra).(Rb) = a^TR^TRb = a.b.*
5. Why did the offset-3 RoPE products differ in the fifth decimal at position 1000? *Answer: float32 rounding of large angles (1000.?) in cos and sin.*

Block 5: Review (1h)

Self-check: - [ ] I can explain why attention is permutation-equivariant. - [ ] I can write the sinusoidal formula and the RoPE rotation from memory. - [ ] I verified that RoPE scores depend only on relative position.

Extra notes: RoPE context extension

Because RoPE encodes position as rotation angles, models trained with context 4k see angles up to 4k.?_i. At 32k, the fast-rotating pairs are fine (they wrapped around many times during training) but the slow pairs reach angles never seen. Position interpolation (Chen et al., 2023) rescales positions so that 32k maps into the trained range, then fine-tunes briefly. Later methods (for example NTK-aware scaling and YaRN) scale the frequencies differently per dimension. You will see these as `rope_scaling` entries in Hugging Face model configs. The lesson is that positional design determines how far a model can be stretched, which matters for Day 115 (long context).

Going deeper (optional)

Read Su et al., "RoFormer: Enhanced Transformer with Rotary Position Embedding" (2021), section 3.

Interview-style question (15 min, written)

"Why do many modern LLMs use RoPE rather than learned absolute positions?" Relative-position behaviour built into the score, no extra parameters, works with KV caching (each key is rotated once), and it can be extended to longer contexts by rescaling.

Common mistakes / how to debug

- Rotating interleaved pairs (0,1),(2,3) in one place and split halves (0..d/2, d/2..d) in another: both are used in practice, but mixing them breaks loaded weights.
- Applying RoPE after the KV cache instead of before, so cached keys have no rotation.
- Forgetting position offsets during cached generation (the new token is at position t, not 0).

What to do next

Day 94 wraps attention into a full transformer block.

Day 94: The transformer block

Review today: Day 93 (+1), Day 91 (+3), Day 87 (+7), Day 73 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

A transformer block has two sub-layers, each wrapped in a residual connection and a normalisation: 1. **Multi-head self-attention** (Day 92): tokens exchange information. 2. **MLP** (also called the feed-forward network): two linear layers with a nonlinearity, expanding to 4.d and back, applied to each position independently. Tokens "think" separately here.

Residual stream. Write the block as x <- x + Attn(LN(x)); x <- x + MLP(LN(x)). The vector x passing through all blocks is the *residual stream*. Each sub-layer reads from it and adds a correction. Gradients flow straight through the additions, which is why dozens of blocks can be trained (the same reason ResNets work, Day 72).

Pre-LN vs post-LN. The original transformer (Vaswani et al., 2017) normalised *after* the residual addition: x <- LN(x + Attn(x)). This post-LN form needs a learning-rate warmup to train stably at depth. GPT-2 moved LayerNorm *before* each sub-layer (pre-LN) and added a final LayerNorm before the output head. Xiong et al. (2020), "On Layer Normalization in the Transformer Architecture", analyse why pre-LN is better behaved early in training. Almost every current decoder LLM is pre-LN, and many replace LayerNorm with RMSNorm (no mean subtraction, no bias).

The MLP. GPT-2 uses GELU. Many current models use a gated variant (SwiGLU: (xW? ? SiLU(xW?))W?) with a hidden size around (8/3).d so that the parameter count stays near 8d^2.

Parameter counting for one block with width d, biases included: - Attention: qkv is d x 3d (+3d bias), output is d x d (+d). Our `_gpt.py` uses no attention biases: 4d^2. - MLP: d x 4d + 4d, then 4d x d + d = 8d^2 + 5d. - Two LayerNorms: 2 x 2d = 4d. At d = 128 that is about 12d^2, the classic rule of thumb for GPT-style models (non-embedding params ~= 12.L.d^2).

GPT on top. Token embedding + position embedding -> L blocks -> final LayerNorm -> linear head to vocabulary logits. *Weight tying* (Press & Wolf, 2017) reuses the embedding matrix as the head, saving vocab.d parameters.

Block 2: Worked example (45 min)

Run `code/week-14/day94_transformer_block.py` (about 20 s). **Verified:** - One block at d = 128: attention **65,536** (= 4d^2), MLP **131,712** (= 8d^2 + 5d), LayerNorms **512**, total **197,760**. - The full GPT (vocab 65, ctx 128, d 128, L 4) has **816,000** parameters, with the head tied to the embedding (**True**). - Gradient flow at initialisation: the ratio of gradient norms of the first block's qkv to the last block's, under a random-projection loss:

depth	pre-LN	post-LN
2	0.99	0.97
12	1.20	0.88
24	1.65	0.98

At this small width and at initialisation, **there is no dramatic difference**. Pre-LN gives the early blocks somewhat *larger* gradients as depth grows. Be wary of a too-simple story about "vanishing gradients in post-LN": the effect described in the literature appears during training with Adam and large learning rates, which the next test probes. - Pre-LN residual stream norm after blocks 1, 4, 8, 12: **8.21, 8.88, 9.73, 10.67**. The stream grows with depth because each block adds to it and nothing renormalises it, which is why the final LayerNorm is needed. - Training stability, 8 blocks, d = 64, 150 steps of Adam **with no warmup** on characters (mean loss over the last 20 steps):

	lr 3e-3	lr 1e-2
pre-LN	2.386	2.355
post-LN	2.382	**3.326**

At the gentle learning rate the two are equal. At the aggressive one, pre-LN still improves while post-LN gets stuck far behind (uniform loss is ln 65 = 4.17, and the bigram baseline is 2.48). That is the practical reason for pre-LN: a wider range of usable learning rates without warmup. One seed per cell, so treat it as a demonstration, not a measurement.

Block 3: Coding lab (2h)

1. (40 min) Write `Block` and `PostLNBlock` yourself and check your parameter counts against the formulas.
2. (40 min) Replace LayerNorm with RMSNorm (x / sqrt(mean(x^2) + eps) . g) and rerun the stability test.
3. (40 min) Replace the GELU MLP with SwiGLU at hidden size 4.d.2/3 and confirm the parameter count stays close to 8d^2.

Block 4: Practice problems (1h)

1. Non-embedding parameters of a 12-layer, d = 768 model by the 12Ld^2 rule? *Answer: 12.12.768^2 ~= 84.9M (GPT-2 small is about 85M non-embedding, 124M with embeddings).*
2. Why does the MLP act per position? *Answer: it is applied independently to each token's vector; only attention mixes positions.*
3. What does weight tying save for vocab 50,257 and d = 768? *Answer: 38.6M parameters.*
4. Why is a final LayerNorm needed in pre-LN? *Answer: the residual stream is never normalised otherwise, and its norm grows with depth (8.2 -> 10.7 here).*
5. Post-LN at lr 1e-2 ended at 3.326. What would you try first? *Answer: a warmup of a few hundred steps, or a lower learning rate.*

Block 5: Review (1h)

Self-check: - [ ] I can write a pre-LN block from memory and count its parameters. - [ ] I can state what the residual stream is. - [ ] I can explain, with my own numbers, why pre-LN is preferred.

Extra notes: reading a block as a circuit

Since each sub-layer *adds* to the residual stream, you can think of the final logits as a sum of contributions from the embeddings and from every attention head and MLP. Interpretability research uses this view: it projects intermediate residual states through the final LayerNorm and head (the "logit lens") to see what the model "would predict" after each block. You can try it on tomorrow's model: after block 1 the predictions are bigram-like; after the last block they use more context.

Going deeper (optional)

Read Xiong et al. (2020) section 3 and compare its analysis with your stability table.

Interview-style question (15 min, written)

"Walk me through one transformer block and its parameter count." Use the formulas above, then mention pre-LN, residuals, weight tying and the 12Ld^2 rule.

Common mistakes / how to debug

- Forgetting the residual: the model trains much worse and deep versions fail.
- Applying the final LayerNorm twice, or forgetting it.
- Using `nn.Linear` biases in some places and not others, then wondering why counts disagree with a paper.

What to do next

Day 95 trains a GPT made of these blocks.

Day 95: Tiny GPT from scratch

Review today: Day 94 (+1), Day 92 (+3), Day 88 (+7), Day 74 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The objective. A GPT is trained to maximise the likelihood of each next token given all previous ones. For a batch of sequences x?...x_T, the input is x?...x_{T-1} and the targets are x?...x_T. Cross-entropy over the vocabulary is averaged over all positions. Because of the causal mask (Day 92), one forward pass gives T training examples per sequence.

Units. Loss is in nats per token. Divide by ln 2 for bits per token; for character models that is bits per character (bpc), a standard way to compare compression. Perplexity is e^loss: the effective number of equally likely choices. A uniform guess over 65 characters gives ln 65 = 4.17 nats.

Baselines first. Before celebrating a transformer, compare it with the simplest model that uses context: a bigram table P(next | current), with add-one smoothing (Day 22 style). If the GPT does not beat it, something is broken.

Training recipe (from `_gpt.train`): AdamW with ? = (0.9, 0.95) and weight decay 0.1, a one-cycle schedule (10% warmup, then cosine-like decay), gradient clipping at norm 1.0, batches of 32 random windows of 128 characters, and init std 0.02. These are the same choices you will see in GPT-style training code, scaled down.

Sampling. Generation feeds the model its own outputs. *Temperature* divides the logits before the softmax (< 1 sharper, > 1 flatter). *Top-k* keeps only the k most likely tokens. Greedy decoding (argmax) tends to loop; sampling gives variety at the risk of nonsense. Our `generate` uses a KV cache (Day 111 explains it). When the cache reaches the context length, it drops the cache and re-reads the last 64 characters, a simple approximation to sliding the window.

Block 2: Worked example (45 min)

Run `code/week-14/day95_tiny_gpt.py` (**88 s** on this box with the CPU otherwise idle, about 46 ms/step; it was many times slower when other jobs shared the CPU). **Verified:** - Vocabulary 65 characters; train 1,003,854 and validation 111,540 tokens. - Uniform loss ln 65 = **4.174**; add-one bigram validation loss **2.482** nats (**3.581** bpc). - GPT: **816,000** parameters; validation loss at init **4.237** (close to uniform, as a good init should be). - Validation loss during training:

step	250	500	750	1000	1250	1500
train	2.298	1.906	1.642	1.541	1.424	1.444
val	2.334	1.998	1.808	1.715	1.645	1.625

- Final validation loss **1.625** nats = **2.344** bpc, against the bigram's 3.581.
- A sample (temperature 0.8, top-k 20, prompt "ROMEO:") has the right *form*: speaker names in capitals followed by a colon, verse-length lines, real words mixed with invented ones ("pressent", "excution"). It even produces "MENENIUS:", a real character from the corpus.

The train and val curves are still close at 1500 steps, so the model is under-fitted, not over-fitted: more steps or a bigger model would help (Day 97).

Block 3: Coding lab (2h)

1. (60 min) Write the GPT yourself (embedding, positions, blocks, final LN, tied head), reusing your Day 94 block, and reproduce a validation loss within 0.05 of 1.625.
2. (30 min) Sample with temperatures 0.5, 0.8, 1.2 and greedy. Describe each.
3. (30 min) Apply the logit lens: after each block, pass the residual through `m.ln` and `m.head` and measure validation loss.

Block 4: Practice problems (1h)

1. Convert 1.625 nats to perplexity. *Answer: e^1.625 ~= 5.08.*
2. Why does a loss at init of 4.237 suggest a correct init? *Answer: it is near ln 65 = 4.174; much higher would mean overconfident random logits.*
3. How many training tokens did the run see? *Answer: 1500 x 32 x 128 = 6,144,000 (about 6 epochs of the training text).*
4. What would make temperature -> 0 equivalent to? *Answer: greedy decoding.*
5. A model's val loss is lower than its train loss early in training. Possible reasons? *Answer: noise from different batches; dropout active only in training; the train number is a single batch.*

Block 5: Review (1h)

Self-check: - [ ] I can convert between nats, bits and perplexity. - [ ] I always compare with a bigram baseline. - [ ] I can explain temperature and top-k.

Extra notes: what CPU scale buys you

At ~46 ms per step, 1500 steps of an 816k-parameter model take a minute and a half. A GPU would run the same steps much faster and, more importantly, let you use a model 10-100x larger in the same time, which is what turns "Shakespeare-shaped nonsense" into readable text. Karpathy's nanoGPT repository trains a similar character-level Shakespeare model as its first example and is a good next reference.

Going deeper (optional)

Read the `model.py` file of nanoGPT (github.com/karpathy/nanoGPT) and list three differences from `_gpt.py`.

Interview-style question (15 min, written)

"How do you know your language model is learning something?" Loss at init near ln V; beats bigram baseline; train and val curves; sample quality; a held-out set.

Common mistakes / how to debug

- Targets not shifted by one: loss goes to nearly zero suspiciously fast.
- Positions beyond the context length during generation: an index error in the position table (this course's first run hit exactly that).
- Evaluating with dropout on.

What to do next

Day 96 swaps characters for subword tokens.

Day 96: BPE from scratch

Review today: Day 95 (+1), Day 93 (+3), Day 89 (+7), Day 75 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why tokenise at all? Day 95's model reads characters, 65 symbols. That gives long sequences: a 128-character context is about 25 English words. Word-level vocabularies fix that but break on unseen words, names, typos and other languages. Subword tokenisation is the compromise used by every major LLM: frequent strings become single tokens, rare strings fall back to smaller pieces, and nothing is ever "unknown".

Byte-pair encoding (Sennrich et al., 2016, adapting a 1994 compression algorithm by Gage). Training: 1. Start from a base vocabulary. Byte-level BPE (GPT-2, Radford et al., 2019) uses the 256 byte values, so any UTF-8 text is representable. 2. Count every adjacent pair of symbols in the corpus. 3. Merge the most frequent pair into a new symbol; record the merge with its rank. 4. Repeat until the vocabulary reaches the target size. Encoding new text replays the merges: repeatedly find the adjacent pair with the *lowest rank* (earliest-learned) and merge it, until no learned pair remains. Decoding just concatenates the bytes of each token.

Pre-tokenisation. Merges are not allowed across arbitrary boundaries. GPT-2 first splits text with a regular expression into words (with their leading space), numbers and punctuation, and runs BPE inside each piece. Our `_bpe.py` uses the simpler pattern `" ?[A-Za-z]+| ?[0-9]+| ?[^A-Za-z0-9\s]+|\s+"`. This has two benefits: tokens like " the" (with a leading space) emerge naturally, and training can count *word types* with frequencies instead of scanning the whole corpus each iteration, which is much faster.

Design choices that matter later. - *Digits.* Our pattern lets digits merge in principle, but Shakespeare has almost no numbers, so they stay single bytes. Several modern tokenizers deliberately split numbers into single digits or groups of three, which affects arithmetic. - *Vocabulary size* trades sequence length against embedding-table size and per-token rarity. - *Training data.* A tokenizer trained on English is inefficient on other scripts: they are split into raw bytes, so the same sentence costs more tokens (and more money in API pricing, and more context).

Other algorithms. WordPiece (BERT) picks merges by a likelihood score rather than raw frequency. Unigram LM (SentencePiece; Kudo, 2018) starts large and prunes. In practice all three produce similar subwords.

Block 2: Worked example (45 min)

Run `code/week-14/day96_bpe.py` (about half a minute plus a one-time tokenizer download). **Verified:** - Training 768 merges (vocabulary 256 + 768 = 1,024) on the 1,003,854-character training split took **15.9 s** in pure Python (20.9 s in an earlier run on a busy CPU). - First merges: ' t', 'he', ' a', 'ou', ' s', ... Last merges: 'ier', 'os', ' nothing', 'ork', 'ichard' (from "Richard": the plays' character names make it into the vocabulary). - Compression: **2.46 bytes/token** on train and **2.28 bytes/token** on validation (111,540 bytes -> 48,838 tokens). Validation is a little worse because the merges were fitted on train. - "To be, or not to be" -> **7**

tokens. "Hello, world! 12345" -> **12** tokens, with the digits split one by one. "naïve café ??" -> **17** tokens: the non-ASCII characters fall back to bytes. All three decode back to the original string exactly. - Validation bytes/token with smaller vocabularies: 300 -> **1.39**, 512 -> **1.90**, 1,024 -> 2.28. Returns diminish as the vocabulary grows. - The pretrained SmolLM2-135M tokenizer (vocabulary 49,152, trained on a large web corpus) gets **3.10 bytes/token** on the same validation text. - Encoder speed on the validation text (quiet machine): Python **72 ms**; Rust (PyO3) serial **7.6 ms**, identical output, about **9x** faster; Rust with rayon across words **5.7 ms**, identical, about **13x**.

The parallel result depends heavily on the machine's state. In an earlier run, while other training jobs were using every core, the same rayon encoder took **42.6 ms**, slower than serial Rust (7.3 ms in that run). The work per word is tiny (a few merges), so when threads have to wait for a free core, the scheduling overhead cancels out the parallelism. Lesson: parallelism pays for big chunks of work on idle cores; measure on the machine you will deploy on.

Block 3: Coding lab (2h)

1. (60 min) Write `train_bpe(text, vocab_size)` yourself from the four steps above, using word counts. Check that your first ten merges match the script's.
2. (30 min) Write `encode` and `decode` and property-test them: for 1,000 random substrings of the validation set, `decode(encode(s)) == s`.
3. (30 min) Build the Rust extension: `cd code/week-14/rust_bpe && maturin develop --release`. Read `src/lib.rs` and find where the GIL is released. Try encoding in batches of 10,000 words with rayon instead of per word.

Block 4: Practice problems (1h)

1. Corpus "aaabdaaabac". What is the first merge? *Answer: "aa" (it occurs 4 times counting overlaps in the naïve count; with non-overlapping replacement, "aa" is still the top pair).*
2. Why use bytes rather than Unicode characters as the base? *Answer: 256 symbols cover every string; no unknown tokens.*
3. Why encode with lowest rank first rather than longest match? *Answer: to reproduce the merge order used in training, so the same text always maps to the same tokens the model saw.*
4. A tokenizer averages 4 bytes/token on English and 1.5 on another language. How much longer is the same content in tokens? *Answer: about 2.7x (if byte lengths are equal; scripts using multi-byte UTF-8 make it worse).*
5. Why might per-digit splitting help arithmetic? *Answer: numbers are represented consistently, so place value is learnable.*

Block 5: Review (1h)

Self-check: - [ ] I can describe BPE training and encoding, and why encoding uses merge ranks. - [ ] I know what pre-tokenisation is for. - [ ] I can explain why my rayon version can be slower than serial Rust on a busy machine.

Extra notes: tokenisation shows up everywhere

Many puzzling LLM behaviours trace back to tokens: counting letters in a word (the model sees " strawberry" as one or a few tokens, not letters), trailing-space sensitivity in prompts, poor arithmetic with irregular number splits, and higher cost for non-English text. When a model fails oddly, print the tokens. On Day 99 we will look at the tokenizer of a real model and its special tokens (begin-of-text, end-of-text, chat-role markers), which are added outside the BPE merges.

Going deeper (optional)

Read the `tokenizers` library documentation on training a byte-level BPE, and compare its speed with ours on the same text.

Interview-style question (15 min, written)

"How does vocabulary size affect a language model?" Larger vocabulary: shorter sequences (more text per context, cheaper attention) but a larger embedding and output layer, and rarer tokens with fewer training examples each. Our runs show diminishing compression gains: 1.39 -> 1.90 -> 2.28 bytes/token from 300 to 1,024.

Common mistakes / how to debug

- Merging across pre-token boundaries, which creates tokens like "e t".
- Encoding by repeatedly applying merges in training order over the whole text (correct, but very slow); use rank-based pair selection per word.

- Decoding tokens one at a time to strings: a single token can be half of a multi-byte UTF-8 character. Concatenate bytes first, then decode.

What to do next

Day 97: how loss improves with model size and data, measured and from the literature.

Day 97: Pretraining objectives and scaling laws

Review today: Day 96 (+1), Day 94 (+3), Day 90 (+7), Day 76 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Pretraining objectives. - *Causal (autoregressive) language modelling:* predict the next token (GPT family; this course). It suits generation directly. - *Masked language modelling:* hide ~15% of tokens and predict them from both sides (BERT; Devlin et al., 2019). It gives strong bidirectional representations for classification and retrieval, but it is not a natural generator. - *Span corruption / denoising:* replace spans with sentinels and generate them (T5; Raffel et al., 2020), used with encoder-decoder models. Today almost all general-purpose LLMs are decoder-only and causal, then adapted by fine-tuning (Week 15).

Compute. A useful approximation: training costs about **6.N.D** floating-point operations for N parameters and D tokens (2ND forward, 4ND backward). Kaplan et al. (2020) use this approximation.

Scaling laws. Kaplan et al. (2020), "Scaling Laws for Neural Language Models", found that test loss falls as a smooth power law in model size N, data size D and compute C, across several orders of magnitude, when the other factors are not the bottleneck. Hoffmann et al. (2022), "Training Compute-Optimal Large Language Models" (the Chinchilla paper), revisited the trade-off with learning-rate schedules matched to each run's length. Their conclusion: for a fixed compute budget, parameters and training tokens should grow in roughly equal proportion, which works out to roughly **20 tokens per parameter**. Their 70B-parameter Chinchilla, trained on 1.4 trillion tokens, outperformed the much larger Gopher at the same compute. The form they fit is L(N, D) = E + A/N^? + B/D^?, where E is the irreducible loss of the data.

How to read these claims. - They describe loss on the training distribution, not any specific capability. - "Compute-optimal" means best loss per *training* FLOP. If a model will serve billions of requests, it is often better to train a smaller model on far more than 20 tokens per parameter, since inference cost scales with N. Many recent open models do this. - Fitted exponents depend on the data, tokenizer and architecture; do not transfer them to your own setting. - Data quality and deduplication shift the whole curve (for example, the FineWeb dataset reports on filtering experiments). Data is now as much an engineering subject as models.

Block 2: Worked example (45 min)

Run `code/week-14/day97_scaling.py` (**35 s**). Five character-level GPTs, each trained for 600 steps of 32 x 64 characters (1,228,800 tokens each). **Verified:**

d	L	non-embedding params	val loss	time
32	1	12,640	2.418	3 s
48	2	56,256	2.278	4 s
64	2	99,584	2.139	4 s
96	3	334,560	2.028	9 s
128	4	791,296	1.954	13 s

Fitted: loss = 3.85 . N^(-0.060) + 0.234, from **five points**. Extrapolated to N = 1e6 and 1e7: 1.922 and 1.705. The largest model saw only **1.55 tokens per parameter**, far below Chinchilla's ~20, so it is data-limited: larger models here are being stopped long before convergence.

Be critical of this fit: five points spanning less than two decades, one seed, a fixed short budget, and an irreducible term (0.234 nats) that is implausibly low for character-level English. It illustrates *the procedure* of scaling-law fitting, not a law. Compare Day 95: the d = 128 model reached 1.625 with 2.5x the steps and twice the context, which shows how much the "data" axis matters.

Block 3: Coding lab (2h)

1. (45 min) Fix the compute instead of the steps: give each model the same 6ND budget (larger models get fewer steps). Which size wins?
2. (45 min) Repeat the sweep with two seeds and add error bars to a log-log plot.
3. (30 min) Compute the FLOPs of Day 95's run by 6ND and divide by its wall time to estimate the CPU's achieved FLOP/s.

Block 4: Practice problems (1h)

1. 6ND for N = 1e9 and D = 2e10? *Answer: 1.2e20 FLOPs.*
2. Chinchilla-optimal tokens for a 1B model? *Answer: about 20B.*
3. Why might a lab train a 1B model on 1T tokens? *Answer: cheaper inference; better loss per serving cost.*
4. Why should the learning-rate schedule be matched to run length when fitting scaling laws? *Answer: a schedule set for a longer run leaves short runs under-trained at their stopping point, biasing the fit; Hoffmann et al. cite this as a reason for their different conclusion.*
5. What is E in L = E + A/N^? + B/D^?? *Answer: the irreducible entropy of the text.*

Block 5: Review (1h)

Self-check: - [ ] I can state the 6ND rule and the ~20 tokens/parameter heuristic, with their sources. - [ ] I can explain why "compute-optimal" is not always "deployment-optimal". - [ ] I can list the weaknesses of my own five-point fit.

Extra notes: data

Pretraining corpora are mostly filtered web text (Common Crawl derivatives), code, books, papers and curated sets. Pipelines deduplicate (exact and near-duplicate), filter by language and quality classifiers, remove personal data and benchmark contamination, and balance domains. Small models such as SmolLM2 report using carefully curated mixtures. When you read a model card, look for the data section: it predicts the model's strengths better than its parameter count.

Going deeper (optional)

Read Hoffmann et al. (2022), Figures 1-3, and Kaplan et al. (2020), section 1.2, then write one paragraph on why their recommendations differ.

Interview-style question (15 min, written)

"You have a fixed GPU budget. How do you pick model size and number of tokens?" Use the Chinchilla trade-off as a starting point, adjust for inference cost and data availability, and run small-scale sweeps to fit your own curve before the big run.

Common mistakes / how to debug

- Comparing models trained for different numbers of tokens as if only size differed.
- Counting embedding parameters in N for small models (Kaplan et al. use non-embedding parameters).
- Extrapolating a fit far beyond its data.

What to do next

Day 98: combine Days 95-96 and train the GPT on BPE tokens.

Day 98: PROJECT W14: train a tiny GPT on BPE tokens; Quiz 14

Review today: Day 97 (+1), Day 95 (+3), Day 91 (+7), Day 77 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Project goal. Train the Day 95 GPT on the Day 96 BPE tokens (vocabulary 1,024) and decide, with a fair metric, whether subword tokens beat characters.

Fair comparison. Loss per token cannot be compared across tokenizers: a BPE token carries ~2.3 characters, so its loss is naturally higher. Convert to **bits per character**: bpc = (loss per token / ln 2) x (tokens / characters). With the same validation text this is a tokenizer-independent measure.

What changes with BPE. - The same 128-position context now covers about 292 characters instead of 128, so the model sees more context. - The embedding table grows from 65 x 128 to 1,024 x 128. - Each token is rarer, so there are fewer training examples per token type and the model can over-fit the training text.

Deliverables. A script (`day98_project_gpt_bpe.py`), a checkpoint (`out/day98_gpt_bpe.pt`), a short report comparing bpc, and one sample. Then take Quiz 14.

Block 2: Worked example (45 min)

Run `code/week-14/day98_project_gpt_bpe.py` (**92 s**). **Verified:** - Encoding: train **408,325** tokens, val **48,838** tokens, in 0.7 s; validation **2.284** characters per token. - GPT: **938,752** parameters, of which the embedding table is 131,072. - Training (1500 steps):

step	500	1000	1500
train	3.694	3.114	2.944
val	3.856	3.550	3.470

- Final: **3.470** nats/token = **2.192 bits/char**, against **2.344** for the character model on Day 95 (same steps, batch and width). The BPE model compresses the validation text about 6% better.
- Its context covers 292 characters vs 128.
- The sample has more real words and longer coherent phrases ("But, my lord, gentle uncle", "With sorrow stark") with fewer broken words than Day 95, though it is still not meaningful.

Note the train/val gap: 2.944 vs 3.470. With 408k training tokens and 1500 x 32 x 128 = 6.1M tokens seen, the model has done about 15 epochs, and it is starting to memorise. The character model had a much smaller gap. For a further gain you would need more data or regularisation (dropout, weight decay), not just more steps.

Block 3: Coding lab (2h)

1. (45 min) Reproduce the run, then add dropout 0.1 and compare the gap and the final bpc.
2. (45 min) Train with vocabulary 512 (retrain the BPE) and plot bpc against vocabulary size: 65 (characters), 512, 1,024.
3. (30 min) Write a 200-word report with the table above and your two additions.

Block 4: Practice problems (1h)

1. Convert 3.470 nats/token at 2.284 characters/token into bpc. *Answer: 3.470 / 0.6931 / 2.284 = 2.192.*
2. Why does the BPE model need fewer steps to see the same amount of text? *Answer: each token covers ~2.3 characters.*
3. Why did the BPE model over-fit more? *Answer: fewer tokens per epoch, so more epochs; more parameters in the embedding.*
4. If val bpc had come out worse for BPE, name two plausible causes. *Answer: too few steps for the larger embedding; over-fitting; context benefit not used.*

Block 5: Review (1h)

Self-check: - [ ] I can compare models with different tokenizers using bpc. - [ ] I can read a train/val gap and pick a remedy. - [ ] I took Quiz 14 and checked my answers.

Extra notes

This model (under 1M parameters, 1M characters of text) is about five orders of magnitude smaller than the SmolLM2-135M model used next week in parameters, and far smaller still in data. Week 15 changes strategy: instead of training from scratch, start from a pretrained model and adapt it.

Going deeper (optional)

Use your Rust encoder to tokenise a larger public-domain text (for example from Project Gutenberg) and retrain to see whether the train/val gap closes.

Interview-style question (15 min, written)

"How would you compare two language models with different tokenizers?" Use bits per character (or per byte) on the same text, never per-token perplexity.

Common mistakes / how to debug

- Comparing per-token losses across tokenizers.
- Encoding validation text with a tokenizer trained on it (leakage into the vocabulary).
- Forgetting to save the tokenizer alongside the model checkpoint.

What to do next

Week 15: Hugging Face models, fine-tuning and LoRA.

Weekly quiz: Week 14

1. Give the shape of the attention score tensor for x of shape (B, T, D) with h heads.
2. What does a padding mask block, and why?
3. What happens to a softmax row that is fully masked?
4. Why is attention without positions permutation-equivariant, and why is that a problem?
5. State the key property of RoPE.
6. Count the parameters of a pre-LN block of width d without attention biases.
7. What did the no-warmup stability test show about post-LN?
8. Convert a loss of 1.625 nats/token on characters to bpc.
9. Why does BPE encoding apply merges by rank?
10. Why did the rayon encoder lose to serial Rust in one run?
11. State the Chinchilla rule of thumb and one reason to train past it.
12. Why must models with different tokenizers be compared in bits per character?

Answers

1. (B, h, T, T).
2. Keys at padding positions, so real tokens' outputs do not depend on how much padding is in the batch.
3. It becomes NaN (0/0).
4. Weights depend only on content, so reordering the inputs only reorders the outputs; word order would be invisible.
5. Rotating q and k by position-dependent angles makes q.k depend only on the relative offset (verified: -0.36481 at offsets of 3 from several starts).
6. 4d^2 + 8d^2 + 5d + 4d = 12d^2 + 9d (197,760 at d = 128).
7. At lr 1e-2 post-LN stalled at loss 3.326 while pre-LN reached 2.355; at 3e-3 both reached about 2.38.
8. 1.625 / ln 2 = 2.344 bpc.
9. To reproduce the training merge order, so the same text gives the tokens the model was trained on.
10. Per-word work is tiny and the cores were busy, so thread scheduling cost more than parallelism saved (42.6 ms vs 7.3 ms).
11. About 20 training tokens per parameter for compute-optimal loss (Hoffmann et al., 2022); train longer to get a smaller model that is cheaper to serve.
12. Per-token loss depends on how much text a token covers; bpc normalises by the text itself.